

LibQueue: Sistema de Gestão de Biblioteca Acadêmica

Ana Clara M. Viana, Charles B. S. Suzart, Emanuel F. Nogueira, Indaiá C. Dutra,
Kaique S. Moreira, Maria Eduarda de O. F. Santos, Pedro S. de Andrade

Curso de Bacharelado em Sistemas de Informação - Instituto Federal da Bahia (IFBA)
CEP 45078 - 900 Vitória da Conquista - BA - Brazil

{202511190007, 202421190015, 202511190004, 202511190047, 202511190021,
202511190026, 202311190041}@ifba.edu.br

Resumo. Este artigo apresenta o desenvolvimento do LibQueue, sistema de gestão de biblioteca escolar para gerenciamento de livros, empréstimos, devoluções e reservas. O sistema foi desenvolvido em Java, utilizando JavaFX e a arquitetura MVC, visando organização e facilidade de manutenção. A modelagem do software foi realizada por meio de diagramas UML e prototipação no Figma. Para o gerenciamento das informações, foram empregadas estruturas de dados adequadas, com destaque para a fila de prioridades utilizada nas reservas, concedendo preferência a professores. Os resultados demonstram maior eficiência na administração do acervo e das atividades da biblioteca. Conclui-se que o LibQueue constitui uma solução viável para a modernização tecnológica de bibliotecas escolares, com potencial para futuras expansões.

Abstract. This article presents the development of LibQueue, a school library management system designed to manage books, loans, returns, and reservations. The system was developed in Java using JavaFX and the Model-View-Controller (MVC) architecture, aiming to provide organization and ease of maintenance. Software modeling was carried out through UML diagrams and interface prototyping in Figma. Appropriate data structures were employed for information management, with emphasis on the priority queue used in reservations, granting preference to teachers. The results demonstrate greater efficiency in managing the library collection and related activities. It is concluded that LibQueue is a viable solution for the technological modernization of school libraries, with potential for future expansions.

1. Introdução

A informatização de bibliotecas escolares tem sido adotada como uma alternativa para melhorar a organização do acervo e a qualidade dos serviços oferecidos aos usuários. Segundo Barbosa e Silva (2019), a automatização contribui para manter o acervo organizado, agilizar buscas e proporcionar melhor acesso às informações. Em bibliotecas que dependem de registros manuais, atividades como controle de empréstimos, devoluções e reservas podem tornar o gerenciamento das informações menos eficiente.

Diante desse cenário, este trabalho apresenta um Sistema de Gestão de Biblioteca Escolar desenvolvido em Java utilizando a biblioteca JavaFX, estruturado segundo os conceitos de orientação a objetos e o padrão Model-View-Controller(MVC). O sistema permite o cadastro e gerenciamento de livros, usuários, reservas, empréstimos e devoluções, proporcionando maior organização e praticidade às atividades da biblioteca. Para sua construção foram utilizados diagramas UML para modelagem do sistema, prototipação das interfaces com a ferramenta Figma e implementação em Java com JavaFX. Além disso, foram empregadas estruturas de dados adequadas para maior eficiência.

O artigo está organizado em: Seção 2 - Trabalhos similares, com a análise comparativa entre o sistema proposto e as plataformas Koha e OpenLibrary; Seção 3 - Desenvolvimento do LibQueue, que detalha as camadas de dados, services, controllers e views, bem como as interfaces do sistema; Seção 4 - Considerações Finais, onde são sintetizados os principais resultados e apontadas direções para trabalhos futuros.

2. Trabalhos Similares

O sistema LibQueue foi comparado com outros gerenciadores de bibliotecas. Um dos escolhidos foi o Koha, de acordo com a Koha Community(2017), este foi o primeiro sistema integrado de gerenciamento de bibliotecas (Sigb) de código aberto, ou seja 100% livre de licenças. O Koha foi desenvolvido em 1999 pela Biblioteca Horowhenua Library Trust da Nova Zelândia.

Com base no manual oficial (IBICT, 2017), verificamos que o Koha é subdividido em 3 seções ou linhas de comando: Usuários e Circulação; Catálogo e Ferramentas Adicionais. É possível fazer uma correlação entre o sistema LibQueue e o Koha. Nota-se a robustez do gerenciador Koha, uma vez que é possível cruzar diversos dados e especificar o perfil dos usuários.

No entanto, apesar do sistema LibQueue ser mais simples, os principais pontos que se destacam frente ao Koha estão relacionados à modalidade de reserva. Enquanto, no Koha isso ocorre de forma Fila Padrão, onde a reserva ocorre a partir da primeira busca do item pesquisado. No LibQueue há uma Fila de Prioridade, onde há um comparativo entre os usuários professor ou aluno e a partir deste comparativo é definido a fila da reserva de determinado livro, dando preferência aos professores em detrimento dos alunos.

Outro ponto relevante é quanto ao aviso de atrasos, enquanto no Koha só é possível o envio de aviso, se previamente for cadastrado no módulo Administração. No LibQueue isso ocorre pela execução em tempo real do método isAtrasado(), ou seja dessa forma não corre o risco do usuário não ser notificado ou de não ter o status de atraso atualizado em tempo real.

Além do Koha, outra plataforma relevante para fins de comparação é o OpenLibrary. Mantido pelo Internet Archive, o OpenLibrary é um projeto de código aberto que tem como objetivo criar uma página digital para cada livro já publicado, funcionando como um amplo catálogo bibliográfico colaborativo em escala global.

A plataforma disponibiliza funcionalidades voltadas à catalogação e à organização de

acervos, permitindo a pesquisa de obras por diferentes critérios, a consulta de metadados bibliográficos, a integração com identificadores padronizados como ISBN e o gerenciamento de empréstimos digitais para determinadas obras. Adicionalmente, o sistema adota uma abordagem colaborativa, possibilitando que usuários contribuam para a atualização e enriquecimento das informações bibliográficas cadastradas.

Em comparação ao LibQueue, observa-se que ambas as soluções compartilham o objetivo de facilitar o acesso e a organização de informações relacionadas a livros. Entretanto, suas finalidades diferem significativamente. Enquanto o OpenLibrary possui abrangência global e atua como um catálogo universal de obras literárias, o LibQueue foi concebido especificamente para atender às necessidades operacionais de bibliotecas escolares. Nesse contexto, o sistema desenvolvido contempla funcionalidades administrativas voltadas ao ambiente educacional, como cadastro de usuários, controle de empréstimos e devoluções, gerenciamento do acervo e administração de filas de reserva com prioridade entre perfis de usuários.

Dessa forma, o OpenLibrary constitui uma importante referência quanto aos mecanismos de catalogação, organização e recuperação de informações bibliográficas, ao passo que o LibQueue concentra seus esforços na gestão eficiente das atividades cotidianas de bibliotecas escolares.

Com base nos estudos apresentados é possível fazer um comparativo entre o sistema LibQueue e os demais trabalhos similares: Koha e OpenLibrary, conforme tabela a seguir:

Tabela 1: Comparativo de funcionalidades entre LibQueue e os sistemas Koha e OpenLibrary

Item	LibQueue	Koha	OpenLibrary
Tipos de Usuários	Aluno, Professor, Bibliotecário.	Múltiplos usuários configuráveis, como por categoria (ex. infantil, adolescente), por tipo (ex.: alunos, professor), por local (ex: escola) dentre outros.	Usuários cadastrados na plataforma para consulta, edição colaborativa de registros bibliográficos e empréstimos digitais quando disponíveis.
Dados necessários para cadastro de usuários	ID (matrícula), Nome, E-mail, Senha e Tipo de Usuário.	Nome e sobrenome, número do cartão (automático - inicia com 1 e vai acrescentando a cada novo usuário), biblioteca de origem e categoria.	Nome de usuário, e-mail e senha para acesso à plataforma e participação colaborativa.
Limite de livros e prazos	Alunos (3 livros / 7 dias); Professores (4 livros / 10 dias); Bibliotecários (5 livros / 10 dias).	Parametrizável de acordo com matriz de regras (configuração da tabela de usuários e circulação).	Não possui regras próprias de circulação física; empréstimos digitais seguem as políticas definidas pelo Internet Archive para cada obra disponível.
Modalidade de	Fila de prioridade (primeiro professor,	Fila padrão (reserva por pesquisa de item, sem especificações de prioridades, seguindo a	Não possui sistema de fila de prioridade para bibliotecas escolares; disponibiliza

reserva	depois aluno).	lógica primeiro a pesquisar).	empréstimos e listas de espera para algumas obras digitais.
Devoluções, Atrasos e Multas	Calcula se houve atraso, comparando a data atual X a data de devolução, então especifica o status, usuário com atraso não pode realizar novos empréstimos ou reservas, fica bloqueado por um período e muda status.	O sistema calcula se há atrasos e envia mensagens para contato do usuário, no entanto para isso ocorrer é necessária a parametrização prévia no módulo Administração, também é possível aplicar multas e gerar e-mails de cobranças.	Não realiza controle de devoluções físicas ou aplicação de multas; os empréstimos digitais possuem prazo automático de expiração ao término do período de empréstimo.
Acervo Bibliográfico	Catalogado com base principal no ISBN atributos (Nome, Autor, ISBN, Gênero).	Catalogado com base no padrão internacional MARC 21 para o registro bibliográfico.	Catálogo bibliográfico colaborativo global, com metadados de obras, autores, edições e integração com identificadores como ISBN.

3. Desenvolvimento do LibQueue

O sistema LibQueue foi implementado na linguagem de programação Java(Oracle, 2026) , adotando o framework JavaFX (OpenJFX, 2026). para a construção de sua interface gráfica. Ambas as tecnologias são integradas por meio da arquitetura MVC (Model-View-Controller), com o objetivo de promover melhor organização, separação de responsabilidades e maior facilidade de manutenção do código.

Nesse contexto, o desenvolvimento do sistema foi estruturado em três frentes principais:

- **Dados:** camada responsável pela modelagem das entidades do sistema e pela implementação das estruturas de dados utilizadas. É composta principalmente pelos pacotes *models* e *repository*, encarregados da organização e da manipulação das informações em memória.
- **Controllers e Services:** atuam como a camada intermediária entre a interface e os dados. Os *services* são responsáveis pela implementação das regras de negócio, enquanto os *controllers* gerenciam a comunicação direta entre o *front-end* e o *back-end*.
- **Views:** encarregadas da construção das interfaces gráficas do sistema, desenvolvidas com o uso de arquivos FXML e da ferramenta Scene Builder(Gluon, 2026).

Além disso, considerando o caráter acadêmico e prototipal do projeto, não foi utilizado um Sistema Gerenciador de Banco de Dados (SGBD). Em vez disso, os dados da aplicação são mantidos em memória por meio de estruturas instanciadas nas classes *DataBaseSeed* e *Biblioteca*. A primeira realiza o povoamento inicial dos dados que serão armazenados na segunda. A classe *Biblioteca*, por sua vez, centraliza os repositórios contendo as listas de

usuários, livros/exemplares do acervo, empréstimos realizados e reservas. Os dados populados consistem em usuários de teste e exemplares de livros em quantidade suficiente para tornar o sistema funcional. Essa abordagem simplificada permitiu maior foco no desenvolvimento da lógica de negócio e na aplicação prática de conceitos fundamentais de estruturas de dados.

3.1 Camada de Dados

A camada de dados do sistema é responsável pela representação das entidades do domínio e pela manipulação das informações em memória. Essa camada é composta pelos pacotes `models`, `repository` e `ed`, que juntos implementam tanto a estrutura dos dados quanto os mecanismos de armazenamento e acesso.

O pacote `models` contém as classes que representam as entidades centrais do sistema, tais como `Usuario`, `Livro`, `Titulo`, `Reserva` e `Emprestimo`. Essas classes encapsulam os atributos e os comportamentos básicos necessários ao funcionamento da aplicação, garantindo organização e coesão na modelagem do domínio.

O pacote `repository` é responsável pela persistência temporária e pela organização dos dados em memória, contendo as classes de acesso aos dados (DAO) e a classe `DataBaseSeed`. Para viabilizar esse armazenamento, foram implementadas estruturas de dados personalizadas, localizadas no pacote `ed`.

As classes do tipo DAO (Data Access Object), como `LivroDAOLista`, `UsuarioDAOLista` e `EmprestimoDAOLista`, encapsulam as operações de inserção, remoção, busca e atualização, promovendo o desacoplamento e a reutilização de código.

A fila de prioridade de reservas foi desenvolvida com base em uma estrutura `PriorityQueue`, associada a um comparador específico (`ReservaComparator`, localizado no pacote `ed`), que define a ordem de atendimento das reservas. Nesse cenário, usuários da categoria "Professor" possuem prioridade estética e funcional em relação a "Alunos". Em casos de empate na mesma categoria, a ordenação é realizada com base na data da reserva, priorizando os registros mais antigos.

Por fim, reitera-se que os dados são inicializados por meio da classe `DataBaseSeed`, que simula um cenário real com um conjunto inicial de informações para testes, eliminando a necessidade de infraestrutura de banco de dados externa.

3.1.1 Classe Título e Fila de Reservas

Embora os dados brutos sejam armazenados na classe `Biblioteca` por meio de listas lineares, o sistema de gerenciamento necessita relacionar de forma direta e eficiente cada obra do acervo com seus respectivos exemplares físicos, empréstimos ativos e reservas pendentes. Inicialmente, para construir a fila de reservas por obra, cogitou-se estruturar uma lista global de filas de prioridade. No entanto, os algoritmos de busca e inserção necessários para manipular essa arquitetura apresentariam alta complexidade computacional. Para solucionar essa limitação e otimizar os relacionamentos, projetou-se a classe `Titulo`.

A classe `Titulo` atua essencialmente como uma estrutura de agrupamento lógico. Cada objeto dessa classe mantém em memória as referências de todos os exemplares de um livro específico, bem como o histórico de empréstimos e a sua respectiva fila de prioridade de reservas.

É importante destacar que a entidade `Titulo` não é persistida de forma direta no repositório; em vez disso, ela é estruturada dinamicamente a cada inicialização do sistema e a cada atualização dos dados. Consequentemente, embora operações de escrita (inserção e

deleção) exijam atualizações tanto na classe Biblioteca quanto na estrutura de Título, as operações de leitura tornam-se consideravelmente mais simples e rápidas, resultando em um ganho significativo de performance para a aplicação. Sob essa ótica, a fila de prioridade encapsulada em cada título armazena estritamente as referências das reservas vinculadas àquela obra específica.

3.2 Camada de Controllers e Services

A camada de controle e serviços é responsável por intermediar o fluxo de informações entre as interfaces de usuário (front-end) e a camada de armazenamento em memória (back-end). Essa divisão é fundamental para garantir o isolamento das regras de negócio, impedindo que a lógica da aplicação fique acoplada aos elementos visuais da interface gráfica.

3.2.1 Pacote Service e Regras de Negócio

O pacote service concentra a implementação de todos os casos de uso e operações que os diferentes perfis de usuários podem realizar no sistema. Para assegurar a coesão e facilitar a manutenção, essa camada foi subdividida em três classes especializadas:

- **AuthService:** concentra a lógica de autenticação e segurança do sistema, sendo responsável pelos procedimentos de validação de login e pelo cadastro de novos usuários.
- **UsuarioService:** gerencia as ações permitidas aos perfis de "Professor" e "Aluno". Suas responsabilidades incluem a recuperação e a visualização do histórico individual de empréstimos ativos e o acompanhamento de reservas associadas ao usuário.
- **BibliotecarioService:** implementa as funcionalidades de nível administrativo, fornecendo ao bibliotecário as ferramentas necessárias para a gestão do acervo (inclusão e baixa de títulos), controle de filas de reserva e registro de devoluções.

Além de executar as regras de negócio, o pacote service atua como um filtro de integridade, manipulando os dados obtidos dos repositórios antes de disponibilizá-los para a exibição.

3.2.2 Pacote Controller e Controle da Interface

Os controllers são responsáveis por capturar as interações do usuário na interface gráfica — como o clique de um botão ou a digitação em um campo de texto — e delegar a execução da respectiva ação para a camada de service.

Nesse ecossistema, adota-se um mapeamento estrito de um para um: existe exatamente uma classe controladora para cada arquivo FXML contido na camada de views. Essa correspondência biunívoca simplifica o rastreamento de eventos, assegurando que as modificações no layout visual de uma tela específica impactem apenas o seu controlador correspondente, reforçando o desacoplamento preconizado pela arquitetura MVC.

Vale salientar que, nas telas voltadas ao usuário comum (como catálogo, lista de reservas e histórico de empréstimos) e nas interfaces de nível administrativo (como controle de reservas, painel de devoluções e inventário), a construção e a renderização dos cartões informativos (cards) são realizadas dinamicamente por meio do respectivo controller, dispensando o uso de estruturas estáticas no arquivo FXML. Essa abordagem foi adotada para garantir que os elementos exibidos na interface gráfica sejam rigorosamente fiéis e sincronizados em tempo real com o estado atual dos dados armazenados na classe Biblioteca, que atua como o motor de persistência em memória da aplicação. Dessa forma, qualquer

alteração no acervo ou no status de algum empréstimo ou reserva reflete-se instantaneamente na interface de maneira reativa e segura.

3.3 Camada de Views

A camada de views compreende a interface gráfica com a qual o usuário interage diretamente, englobando todas as telas que compõem o sistema LibQueue. Desenvolvida utilizando a linguagem de marcação FXML em conjunto com a ferramenta Scene Builder, essa camada tem como função exclusiva a disposição visual dos elementos, delegando qualquer processamento lógico aos seus respectivos controladores.

As interfaces são atualizadas de forma dinâmica e reativa, refletindo imediatamente na tela as modificações de estado decorrentes das ações do usuário e das validações de regras de negócio.

Um diferencial arquitetural importante na construção dessas views diz respeito às telas destinadas à exibição de listagens de dados (como catálogos, filas de espera e históricos). Em vez de adotar componentes estáticos ou tabelas rígidas, os arquivos FXML dessas páginas foram projetados contendo containers flexíveis de layout (notadamente o FlowPane). Esses containers funcionam como "molduras" ou espaços receptores vazios mapeados no código. Durante a execução do sistema, o respectivo controller renderiza os cartões informativos (cards) de maneira isolada e os injeta dinamicamente dentro desses espaços, garantindo um layout fluido, responsivo e fiel aos dados em memória.

3.4 Demonstração das Interfaces do Sistema

Para demonstrar o comportamento visual e os mecanismos de usabilidade do LibQueue, as interfaces foram agrupadas em três macrofluxos baseados em casos de uso e níveis de permissão de acesso.

3.4.1 Interfaces de Autenticação (Fluxo de Acesso)

O ecossistema de acesso é unificado, utilizando uma interface inteligente de entrada. A camada de Authservice realiza uma verificação do tipo de usuário através da senha e da matrícula no momento do clique no botão de redirecionamento. A figura 2 ilustra o fluxo de autenticação e registro.

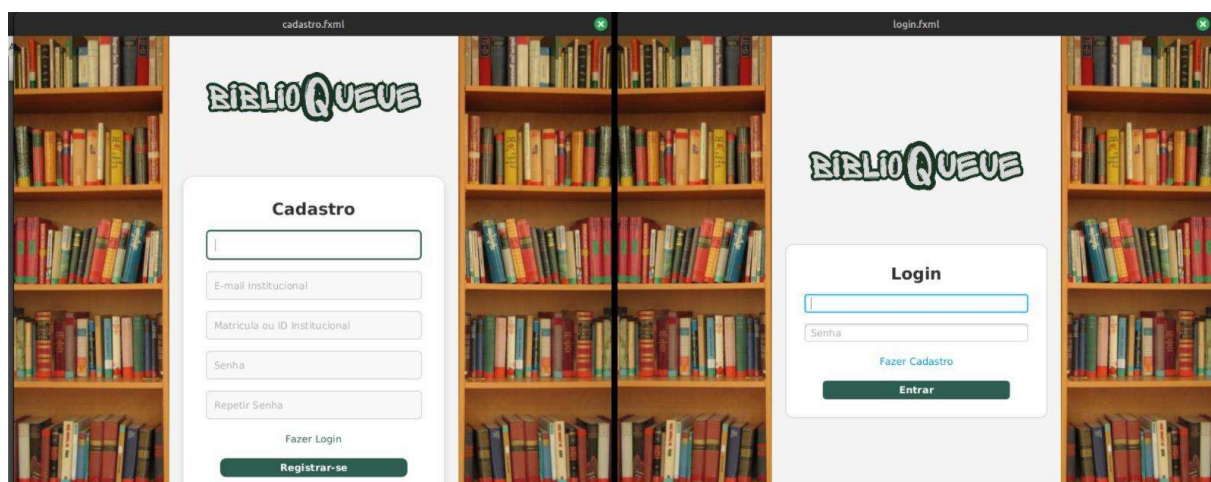


Figura 1. Tela de cadastro e login

3.4.2 Painel Administrativo do Bibliotecário

Por possuir privilégios elevados, a área do bibliotecário foi segmentada em quatro módulos focados na eficiência operacional intuitiva, conforme figura 3.

O Dashboard centraliza os módulos principais de visualização do bibliotecário (Figura 3A); o inventário reúne as amostras disponíveis para empréstimos (Figura 3B); a fila de espera reúne os usuários aguardando disponibilidade de exemplares (Figura 3C); o painel de devoluções permite o bibliotecário registrar devoluções de exemplares ativos (Figura 3D).

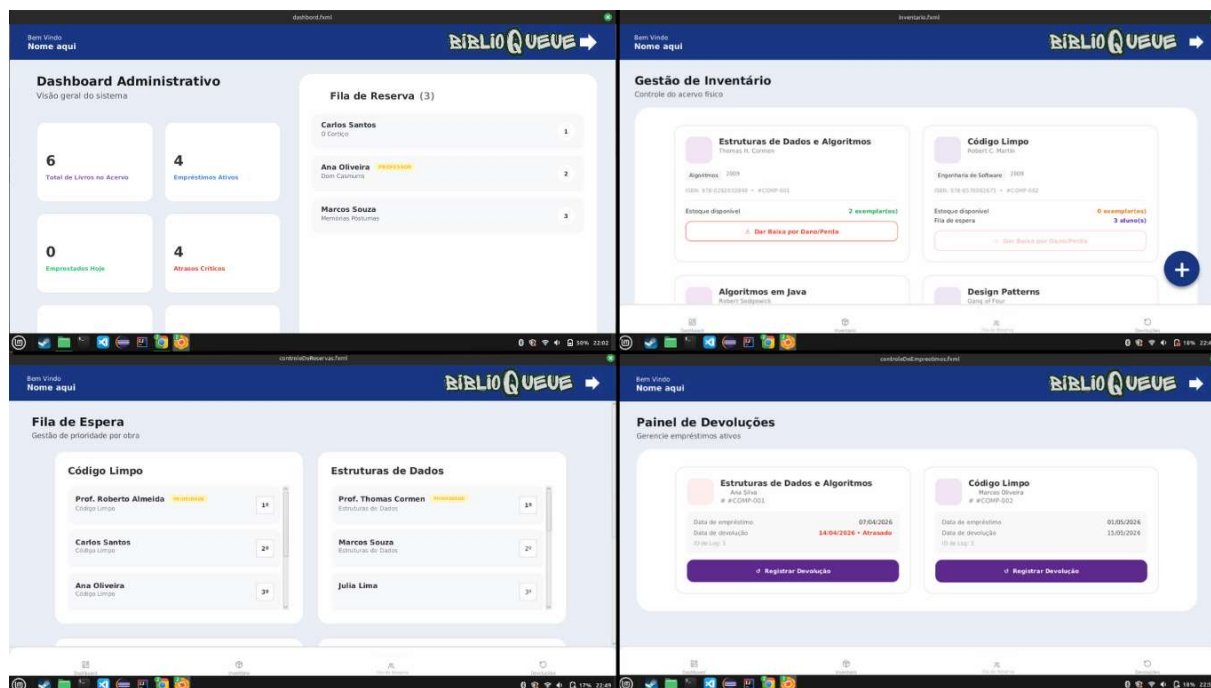


Figura 2. Telas de bibliotecário

3.4.3 Painel de Usuários

Nestes módulos os alunos têm acesso aos livros das disciplinas em que estão matriculados. Os livros podem ser separados por categorias e podem ser pegos a qualquer momento caso o livro esteja disponível, caso não esteja ele pode entrar na fila de espera.

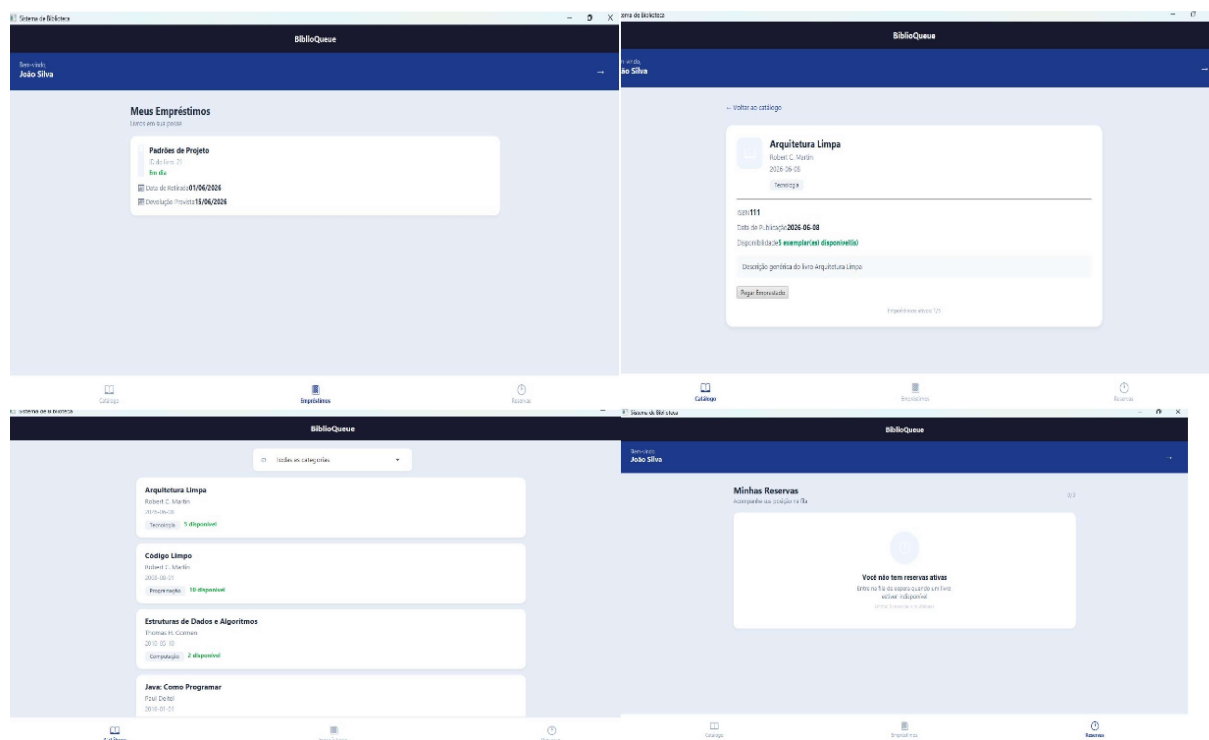


Figura 3. Telas do Catálogo e Empréstimos de Livros

4. Considerações Finais

O desenvolvimento do sistema LibQueue permitiu preencher lacunas operacionais comumente identificadas em sistemas tradicionais de gerenciamento de acervos, como o Koha, ao introduzir uma solução nativa para o gerenciamento de filas reserva com critérios de prioridade entre os perfis dos usuários professor e aluno. A adoção da arquitetura MVC em conjunto com o framework JavaFX na camada de views, provou-se eficiente ao garantir o desacoplamento do código e viabilizar uma atualização de interface gráfica reativa e síncrona com as manipulações de dados em memória.

As decisões de engenharia de software utilizadas, em especial, a abstração fornecida pela classe Título e o tratamento de atrasos em tempo real pelo método `isAtrasado()`, otimizaram a complexidade computacional de buscas, compensando de forma estruturada a ausência de um Sistema de Banco de Dados nesta fase de protótipo. Sob uma perspectiva acadêmica, o projeto demonstrou a viabilidade prática e a utilização de conceitos de estrutura de dados (como a `PriorityQueue` aplicada ao ordenamento de reservas) em cenários acadêmicos reais de instituições educacionais.

Como melhoria futura do projeto, aponta-se uma migração para um sistema de Banco de Dados para garantir a persistência e integridade dos dados a longo prazo. Ademais, projeta-se o desenvolvimento de um módulo automatizado de comunicação externa para o envio de notificações e avisos de pendências via APIs de e-mail ou mensagens instantâneas, além de uma integração com APIs globais de busca de metadados, como a plataforma OpenLibrary.

Referências

BARBOSA, E. T.; SILVA, A. O. Implantação de software de gestão de biblioteca escolar na rede municipal de ensino de Vila Velha - ES. In: CONGRESSO BRASILEIRO DE BIBLIOTECONOMIA, DOCUMENTAÇÃO E CIÊNCIA DA INFORMAÇÃO, 28., 2019, Vitória. Anais [...]. Vitória: FEBAB, 2019. Disponível em:

<https://portal.febab.org.br/cbbd2019/article/view/2314>. Acesso em: 31 maio 2026.

IBICT (2017) “Guia do usuário do Koha”, <https://livroaberto.ibict.br/handle/123456789/1064>. Acesso em: 31 maio 2026.

OPEN LIBRARY. Open Library. 2026. Disponível em: <https://openlibrary.org> . Acesso em: 31 maio 2026.

Oracle. Java Platform, Standard Edition Documentation. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 16 jun. 2026.

OpenJFX. OpenJFX Documentation. Disponível em: <https://openjfx.io/>. Acesso em: 16 jun. 2026.

GLUON. JavaFX Scene Builder Documentation. Disponível em: <https://gluonhq.com/products/scene-builder/>. Acesso em: 16 jun. 2026.